

Universidad Anáhuac México

Facultad de Ingeniería

Guía de Ejemplos

Algoritmos y Programación en Python

Cinco ejemplos comentados — formato enunciado / solución

- Ciclos **for** anidados
- Módulo **random**
- Ordenamiento de listas y matrices
- Ciclos **while** y **while True**
- Sentencia **match/case** (switch-case)

Prof. DSc. Aboud Barsekh-Onji

Facultad de Ingeniería | Universidad Anáhuac México

Ciclo escolar 2025-2026

0 Índice

1. Ejemplo 1 — Ciclos for anidados	2
2. Ejemplo 2 — Módulo random	5
3. Ejemplo 3 — Ordenamiento de listas y matrices	8
4. Ejemplo 4 — Ciclos while y while True	11
5. Ejemplo 5 — Sentencia match/case	14
Resumen comparativo de estructuras	17

1 Ejemplo 1 — Ciclos **for** anidados

Objetivo

Dominar la estructura de ciclos **for** anidados para recorrer estructuras bidimensionales, construir patrones visuales y acumular resultados fila por fila.

Conceptos necesarios

- Un ciclo **for** anidado es un ciclo dentro de otro ciclo.
- Por cada iteración del ciclo **externo**, el ciclo **interno** se ejecuta *completo*.
- El número total de iteraciones es el *producto* de los rangos de ambos ciclos.
- `print(..., end=)` imprime sin salto de línea al final.
- `len(lista)` devuelve el número de elementos de una lista.

Enunciado — `ejemplo1.py`

Escribe un programa Python que realice las tres tareas siguientes usando **únicamente** ciclos **for** anidados:

1. **Tabla de multiplicar.** Imprime en pantalla la tabla de multiplicar del 1 al 5 en formato de cuadrícula. Cada celda debe mostrar la operación ***i* × *j* = resultado**. Usa **`end=`** para mantener cada fila en una sola línea.
2. **Triángulo de asteriscos.** Imprime un triángulo rectángulo de asteriscos de **6** filas. La fila *k* debe contener exactamente *k* asteriscos separados por espacios.
3. **Suma por filas de una matriz 3×3 .** Dada la matriz:

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Recorre la matriz con ciclos **for** anidados, imprime los elementos de cada fila y calcula la suma de cada una.

Solución — Ejemplo 1: Ciclos **for** anidados

ejemplo1.py

```

1 # =====
2 # ejemplo1.py - Ciclos FOR anidados
3 # Materia: Algoritmos y Programación en Python
4 # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5 # =====
6
7 # --- Parte 1: Tabla de multiplicar ---
8 print("=" * 40)
9 print(" TABLA DE MULTIPLICAR (1 al 5)")
10 print("=" * 40)
11
12 for i in range(1, 6):
13     for j in range(1, 6):
14         print(f"{i} x {j} = {i * j:<4}", end=" ")
15     print() # salto de línea al terminar cada fila
16
17 # --- Parte 2: Patrón de asteriscos ---
18 print("\n" + "=" * 40)
19 print(" TRIÁNGULO DE ASTERISCOS")
20 print("=" * 40)
21
22 filas = 6
23 for fila in range(1, filas + 1):
24     for col in range(fila):
25         print("*", end=" ")
26     print()
27
28 # --- Parte 3: Suma de elementos de una matriz 3x3 ---
29 print("\n" + "=" * 40)
30 print(" SUMA DE FILAS EN UNA MATRIZ 3x3")
31 print("=" * 40)
32
33 matriz = [
34     [1, 2, 3],
35     [4, 5, 6],
36     [7, 8, 9]
37 ]
38
39 for i in range(len(matriz)):
40     suma_fila = 0
41     for j in range(len(matriz[i])):
42         suma_fila += matriz[i][j]
43     print(f"{matriz[i][j]:3}", end=" ")
44     print(f" -> suma = {suma_fila}")

```

Listing 1: ejemplo1.py — For anidados

Parte 1 — Tabla de multiplicar. El ciclo exterior itera **i** de 1 a 5 (filas); el interior itera **j** también de 1 a 5 (columnas). El argumento **end=** evita el salto de línea para imprimir en la misma fila; el **print()** vacío al final de la iteración exterior produce el retorno de carro.

Parte 2 — Triángulo de asteriscos. El ciclo exterior controla el número de filas. El interior imprime tantos asteriscos como indique **fila**. La variable del ciclo exterior actúa como límite del interior: cuando **fila=1** se imprime 1 asterisco, cuando **fila=6** se imprimen 6.

Parte 3 — Suma por filas de una matriz. `len(matriz)` da el número de filas y `len(matriz[i])` el número de columnas de la fila *i*. La variable **suma_fila** se reinicia a cero en cada iteración del ciclo exterior antes de acumular los elementos de esa fila.

Salida esperada

```

1 x 1 = 1      1 x 2 = 2      1 x 3 = 3      1 x 4 = 4      1 x 5 = 5
...
5 x 1 = 5      5 x 2 = 10     5 x 3 = 15     5 x 4 = 20     5 x 5 = 25

*
* *
* * *
* * * *
* * * * *
* * * * * *

1  2  3  -> suma = 6
4  5  6  -> suma = 15
7  8  9  -> suma = 24

```

2 Ejemplo 2 — Módulo `random`

Objetivo

Aprender a generar valores aleatorios, seleccionar y mezclar listas, y construir simulaciones sencillas usando el módulo estándar `random`.

Conceptos necesarios

- `import random` — importar el módulo de aleatoriedad.
- `random.randint(a, b)` — entero aleatorio en $[a, b]$ inclusive.
- `random.random()` — flotante uniforme en $[0, 0, 1, 0)$.
- `random.uniform(a, b)` — flotante en $[a, b]$.
- `random.choice(seq)` — elemento aleatorio de una secuencia.
- `random.shuffle(seq)` — mezcla la lista *en su lugar* (modifica el original).
- `random.sample(seq, k)` — muestra sin reposición de tamaño k .

Enunciado — `ejemplo2.py`

Escribe un programa Python que demuestre las principales funciones del módulo `random` en tres partes:

1. **Generación de números aleatorios.** Genera e imprime: un entero aleatorio entre 1 y 100, un flotante en $[0, 0, 1, 0)$ con 4 decimales, y un flotante en $[5, 0, 10, 0]$ con 4 decimales.
2. **Listas y elección aleatoria.** Dada la lista `['rojo', 'azul', 'verde', 'amarillo', 'morado']`:
 - Elige *un* color al azar con `random.choice`.
 - Mezcla la lista completa con `random.shuffle` e imprime el resultado.
 - Obtén una muestra de 3 colores distintos con `random.sample`.
3. **Simulación de dados.** Simula **10 lanzamientos** de un dado de 6 caras. Muestra el resultado de cada lanzamiento e imprime al final un histograma de barras con caracteres `|` que indique cuántas veces salió cada cara.

Solución — Ejemplo 2: Módulo random

ejemplo2.py

```
1 # =====
2 # ejemplo2.py - Módulo RANDOM
3 # Materia: Algoritmos y Programación en Python
4 # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5 # =====
6
7 import random
8
9 # --- Parte 1: Números aleatorios básicos ---
10 print("=" * 45)
11 print("  NÚMEROS ALEATORIOS BÁSICOS")
12 print("=" * 45)
13
14 print(f"Entero entre 1 y 100:      {random.randint(1, 100)}")
15 print(f"Flotante entre 0.0 y 1.0:  {random.random():.4f}")
16 print(f"Flotante entre 5.0 y 10.0: {random.uniform(5.0, 10.0):.4f}")
17
18 # --- Parte 2: Lista aleatoria y elección ---
19 print("\n" + "=" * 45)
20 print("  LISTAS Y ELECCIÓN ALEATORIA")
21 print("=" * 45)
22
23 colores = ["rojo", "azul", "verde", "amarillo", "morado"]
24 print(f"Lista original: {colores}")
25
26 color_elegido = random.choice(colores)
27 print(f"Color elegido: {color_elegido}")
28
29 random.shuffle(colores)
30 print(f"Lista mezclada: {colores}")
31
32 muestra = random.sample(colores, 3)
33 print(f"Muestra de 3: {muestra}")
34
35 # --- Parte 3: Simulación de dados (10 lanzamientos) ---
36 print("\n" + "=" * 45)
37 print("  SIMULACIÓN: 10 LANZAMIENTOS DE DADO")
38 print("=" * 45)
39
40 conteo = {i: 0 for i in range(1, 7)}
41
42 for lanzamiento in range(1, 11):
43     resultado = random.randint(1, 6)
44     conteo[resultado] += 1
45     print(f"  Lanzamiento {lanzamiento:2}: {resultado}")
46
47 print("\nFrecuencia de cada cara:")
48 for cara, veces in conteo.items():
49     print(f"  Cara {cara}: {'| ' * veces} ({veces})")
```

Listing 2: ejemplo2.py — Módulo random

Parte 1 — Generación básica. Se muestran las tres funciones de generación de números: entera (`randint`), flotante uniforme en $[0, 1)$ (`random()`) y flotante en rango arbitrario (`uniform`). El especificador `:.4f` en el f-string limita la salida a cuatro decimales.

Parte 2 — Listas y elección. `random.choice` selecciona *un* elemento sin modificar la lista. `shuffle` reordena la lista **en memoria** (el original cambia). `sample` devuelve una *nueva* lista con k elementos distintos del original.

Parte 3 — Simulación de dados. Se usa un diccionario de conteos inicializado con `{i: 0 for i in range(1,7)}` y un ciclo `for` para acumular los resultados. La visualización con `|` es proporcional a la frecuencia observada.

Salida esperada (fragmento — los números varían por ser aleatorios)

```
Entero entre 1 y 100:      37
Flotante entre 0.0 y 1.0:  0.4218
Color elegido:    verde
Lista mezclada:    ['morado', 'rojo', 'amarillo', 'azul', 'verde']
Lanzamiento 1: 3
Cara 3:  || (2)
Cara 6:  |||| (4)
```

3 Ejemplo 3 — Ordenamiento de listas y matrices

Objetivo

Implementar algoritmos clásicos de ordenamiento (burbuja y selección), comprender su complejidad $O(n^2)$, y aplicar `sorted()` para ordenar estructuras de datos complejas mediante funciones `lambda`.

Conceptos necesarios

- **Bubble Sort:** compara pares adyacentes e intercambia si están en orden incorrecto; requiere $n - 1$ pasadas.
- **Selection Sort:** en cada pasada selecciona el mínimo del subarray no ordenado y lo coloca en su posición.
- **Complejidad:** ambos son $O(n^2)$ en tiempo y $O(1)$ en espacio.
- `lista[:]` — copia superficial de una lista.
- `a, b = b, a` — intercambio en una sola línea (swap).
- `sorted(iterable, key=f, reverse=True)` — ordena sin modificar el original.
- `lambda x: expr` — función anónima de una línea.

Enunciado — ejemplo3.py

Trabaja con una lista de 10 enteros generada con `random.seed(42)`.

1. **Bubble Sort.** Implementa el algoritmo de ordenamiento burbuja *desde cero* usando dos ciclos `for` anidados. Imprime la lista original y la ordenada.
2. **Selection Sort.** Implementa el algoritmo de ordenamiento por selección *desde cero*. Usa una variable `idx_min` para guardar la posición del menor elemento en cada pasada. Imprime la lista original y la ordenada.
3. **Ordenamiento de alumnos por promedio.** Dada la siguiente lista de alumnos con sus tres calificaciones:

Ana [85, 90, 78]	Luis [70, 65, 80]	Sofía [95, 92, 98]	Carlos [60, 75, 70]	Elena [88, 84, 91]
------------------	-------------------	--------------------	---------------------	--------------------

Calcula el promedio de cada alumno, agrégalo al diccionario y usa `sorted()` con `lambda` para ordenarlos de mayor a menor promedio. Imprime la tabla con nombre, notas y promedio.

Solución — Ejemplo 3: Ordenamiento

ejemplo3.py

```

1  # =====
2  # ejemplo3.py - Ordenamiento de listas y matrices
3  # Materia: Algoritmos y Programación en Python
4  # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5  # =====
6
7  import random
8
9  # Generamos una lista aleatoria para ordenar
10 random.seed(42)
11 numeros = [random.randint(1, 99) for _ in range(10)]
12
13 # --- Parte 1: Ordenamiento burbuja (Bubble Sort) ---
14 print("=" * 45)
15 print(" ORDENAMIENTO BURBUJA (BUBBLE SORT)")
16 print("=" * 45)
17
18 burbuja = numeros[:] # copia de la lista original
19 print(f"Original: {burbuja}")
20
21 n = len(burbuja)
22 for i in range(n - 1):
23     for j in range(n - 1 - i):
24         if burbuja[j] > burbuja[j + 1]:
25             burbuja[j], burbuja[j + 1] = burbuja[j + 1], burbuja[j]
26
27 print(f"Ordenada: {burbuja}")
28
29 # --- Parte 2: Ordenamiento por selección (Selection Sort) ---
30 print("\n" + "=" * 45)
31 print(" ORDENAMIENTO POR SELECCIÓN")
32 print("=" * 45)
33
34 seleccion = numeros[:]
35 print(f"Original: {seleccion}")
36
37 n = len(seleccion)
38 for i in range(n):
39     idx_min = i
40     for j in range(i + 1, n):
41         if seleccion[j] < seleccion[idx_min]:
42             idx_min = j
43     seleccion[i], seleccion[idx_min] = seleccion[idx_min], seleccion[i]
44
45 print(f"Ordenada: {seleccion}")
46
47 # --- Parte 3: Ordenar una matriz de calificaciones por promedio ---
48 print("\n" + "=" * 45)
49 print(" ORDENAR ALUMNOS POR PROMEDIO")
50 print("=" * 45)
51
52 alumnos = [
53     {"nombre": "Ana", "notas": [85, 90, 78]},
54     {"nombre": "Luis", "notas": [70, 65, 80]},

```

```

55     {"nombre": "Sofía", "notas": [95, 92, 98]},
56     {"nombre": "Carlos", "notas": [60, 75, 70]},
57     {"nombre": "Elena", "notas": [88, 84, 91]},
58 ]
59
60 for alumno in alumnos:
61     alumno["promedio"] = sum(alumno["notas"]) / len(alumno["notas"])
62
63 alumnos_ordenados = sorted(alumnos, key=lambda a: a["promedio"], reverse=
    True)
64
65 print(f'{{'Nombre':<10}} {{'Notas':<20}} {{'Promedio':>8}}")
66 print("-" * 45)
67 for a in alumnos_ordenados:
68     print(f'{{a['nombre']:<10}} {{str(a['notas']):<20}} {{a['promedio']:>8.2
    f}}")

```

Listing 3: ejemplo3.py — Ordenamiento

Parte 1 — Bubble Sort. El ciclo exterior limita las pasadas a $n - 1$; el interior llega hasta $n - 1 - i$ porque los últimos i elementos ya están en su posición final. El intercambio se realiza en una sola línea con asignación múltiple: **a, b = b, a**.

Parte 2 — Selection Sort. `idx_min` guarda el índice del menor elemento encontrado en el subarray $[i, n)$. Al terminar el ciclo interior, se intercambia el elemento en posición i con el mínimo encontrado.

Parte 3 — Sorted con lambda. `sorted()` recibe `key=lambda a: a["promedio"]` para ordenar la lista de diccionarios por valor numérico. `reverse=True` invierte el orden (de mayor a menor). El original no se modifica.

Salida esperada

```

Original:  [82, 15, 4, 95, 36, 32, 29, 18, 95, 14]
Ordenada:  [4, 14, 15, 18, 29, 32, 36, 82, 95, 95]

```

Nombre	Notas	Promedio
Sofia	[95, 92, 98]	95.00
Elena	[88, 84, 91]	87.67
Ana	[85, 90, 78]	84.33

4 Ejemplo 4 — Ciclos **while** y **while True**

Objetivo

Distinguir entre el ciclo **while** con condición explícita y el patrón **while True** con **break/continue**, aplicándolos en contextos numéricos y de control de flujo.

Conceptos necesarios

- **while condición:** — ejecuta el cuerpo mientras la condición sea verdadera.
- **while True:** — ciclo potencialmente infinito; requiere **break**.
- **break** — termina inmediatamente el ciclo más interno.
- **continue** — salta el resto del cuerpo y vuelve a evaluar la condición.
- **abs(x)** — valor absoluto de x .
- Método de Newton-Raphson para $\sqrt{S}$:

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{S}{x_n} \right)$$

Enunciado — ejemplo4.py

Escribe un programa Python con tres partes que demuestren el uso de **while** y **while True**:

1. **Potencias de 2 con **while**.** Usando *solo* un ciclo **while**, imprime todas las potencias de 2 desde $2^0 = 1$ hasta $2^9 = 512$. Muestra tanto el exponente como el resultado.
2. **Raíz cuadrada con Newton-Raphson y **while**.** Implementa el método iterativo de Newton-Raphson para calcular $\sqrt{50}$. La estimación inicial es $x_0 = 25,0$. El ciclo debe terminar cuando el error absoluto $|x^2 - 50| < 10^{-6}$. Imprime el resultado con 8 decimales y el número de iteraciones realizadas.
3. **Juego de adivinar con **while True**.** Un número secreto aleatorio entre 1 y 20 es elegido por el programa. Usando **while True** y búsqueda binaria automática (el programa adivina solo):
 - Mantén variables **bajo** y **alto** para el rango de búsqueda.
 - En cada intento calcula **intento = (bajo + alto) // 2**.
 - Usa **break** si acierta o si se superan 10 intentos.
 - Usa **continue** para ajustar **bajo** o **alto** y repetir.

Solución — Ejemplo 4: while y while True

ejemplo4.py

```

1  # =====
2  # ejemplo4.py - Ciclos WHILE y WHILE TRUE
3  # Materia: Algoritmos y Programación en Python
4  # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5  # =====
6
7  import random
8
9  # --- Parte 1: While con contador ---
10 print("=" * 45)
11 print("  WHILE: POTENCIAS DE 2")
12 print("=" * 45)
13
14 potencia = 1
15 n = 0
16 while potencia <= 512:
17     print(f"  2^{n:2} = {potencia}")
18     n += 1
19     potencia *= 2
20
21 # --- Parte 2: While con condición de convergencia ---
22 print("\n" + "=" * 45)
23 print("  WHILE: ALGORITMO DE LA RAÍZ CUADRADA")
24 print("  (Método de Newton-Raphson)")
25 print("=" * 45)
26
27 objetivo = 50.0
28 x = objetivo / 2.0          # estimación inicial
29 tolerancia = 1e-6
30 iteracion = 0
31
32 while abs(x * x - objetivo) > tolerancia:
33     x = (x + objetivo / x) / 2.0
34     iteracion += 1
35
36 print(f"  sqrt({objetivo}) ≈ {x:.8f} (en {iteracion} iteraciones)")
37 print(f"  Verificación: {x:.8f}² = {x*x:.8f}")
38
39 # --- Parte 3: While True con break y continue ---
40 print("\n" + "=" * 45)
41 print("  WHILE TRUE: JUEGO DE ADIVINAR EL NÚMERO")
42 print("  (modo demo: el programa 'adivina' solo)")
43 print("=" * 45)
44
45 secreto = random.randint(1, 20)
46 intentos = 0
47 limite = 10
48 bajo, alto = 1, 20
49
50 print(f"  Número secreto oculto entre 1 y 20")
51
52 while True:
53     if intentos >= limite:
54         print(f"  Se agotaron los {limite} intentos. El número era {

```

```

55     secreto}).")
56         break
57
58     intento = (bajo + alto) // 2    # búsqueda binaria automática
59     intentos += 1
60
61     if intento == secreto:
62         print(f" Intento {intentos}: {intento} -> ¡CORRECTO! Encontrado
63         en {intentos} intentos.")
64         break
65     elif intento < secreto:
66         print(f" Intento {intentos}: {intento} -> muy bajo")
67         bajo = intento + 1
68         continue
69     else:
70         print(f" Intento {intentos}: {intento} -> muy alto")
71         alto = intento - 1
72         continue

```

Listing 4: ejemplo4.py — While y While True

Parte 1 — Potencias de 2. La condición `while potencia <= 512` se evalúa *antes* de cada iteración. Cuando **potencia** supera 512, el ciclo termina naturalmente. Tanto **n** como **potencia** se actualizan dentro del cuerpo.

Parte 2 — Newton-Raphson. El ciclo continúa mientras el error absoluto $|x^2 - S|$ supere la tolerancia. No se conoce de antemano cuántas iteraciones serán necesarias: el **while** con condición de convergencia es la estructura natural para este caso.

Parte 3 — While True con búsqueda binaria. `while True` crea un ciclo indefinido. El primer **break** (acierto o límite de intentos) garantiza la terminación. **continue** omite el resto del cuerpo y vuelve al inicio del ciclo, separando limpiamente los tres casos: igual, menor y mayor.

Salida esperada (fragmento)

```

2^ 0 = 1
2^ 9 = 512
sqrt(50.0) ≈ 7.07106781 (en 6 iteraciones)
Verificacion: 7.07106781^2 = 50.00000000
Intento 1: 10 -> muy alto
Intento 2: 5 -> Correcto! Encontrado en 2 intentos.

```

5 Ejemplo 5 — Sentencia `match/case`

Objetivo

Usar la sentencia `match/case` de Python 3.10+ como alternativa estructurada a cadenas de `if/elif/else`, aprovechando guardas y el patrón comodín.

Conceptos necesarios

- `match expr`: evalúa la expresión y la compara con cada `case`.
- El primer `case` que coincida se ejecuta; los demás se ignoran.
- `case _`: es el comodín (equivalente a `default` en C/Java).
- **Guarda**: `case n if condición`: añade un filtro booleano al patrón.
- Disponible desde **Python 3.10**; el entorno **conda research** usa 3.11.

Nota de compatibilidad

La sentencia `match/case` requiere Python ≥ 3.10 . En versiones anteriores se usa `if/elif/else` o un diccionario de funciones.

Enunciado — `ejemplo5.py`

Escribe un programa Python con tres funciones que usen `match/case`:

1. **Calculadora simple.** Define `calculadora(a, operacion, b)` que soporte los operadores `+`, `-`, `*`, `/`, `**`. El caso `/` debe incluir validación de división entre cero. El caso comodín debe devolver un mensaje de error. Prueba la función con al menos 7 operaciones, incluyendo división entre cero.
2. **Clasificación de figuras geométricas.** Define `clasificar_figura(lados)` que devuelva el nombre de la figura según su número de lados (3 a 6). Para $n > 6$ usa una **guarda**: `case n if n > 6`: y retorna `f"Polígono de {n} lados"`. Prueba con los valores: 3, 4, 5, 6, 8, 12, 1.
3. **Despachador de comandos de texto.** Define `ejecutar_comando(cmd)` que procese los comandos: `inicio`, `ayuda`, `salir`, `config`, `datos`. El comodín debe sugerir escribir `ayuda`. Simula la ejecución de la lista: `[inicio, .ayuda, "salir", config, "datos", "desconocido"]`.

Solución — Ejemplo 5: match/case

ejemplo5.py

```

1  # =====
2  # ejemplo5.py - MATCH / CASE (Switch-Case en Python 3.10+)
3  # Materia: Algoritmos y Programación en Python
4  # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5  # =====
6
7  # --- Parte 1: Calculadora simple con match-case ---
8  print("=" * 45)
9  print("  CALCULADORA SIMPLE")
10 print("=" * 45)
11
12 def calculadora(a, operacion, b):
13     match operacion:
14         case "+":
15             return a + b
16         case "-":
17             return a - b
18         case "*":
19             return a * b
20         case "/":
21             if b == 0:
22                 return "Error: división entre cero"
23             return a / b
24         case "**":
25             return a ** b
26         case _:
27             return f"Operación '{operacion}' no reconocida"
28
29 operaciones = [(10, "+", 5), (10, "-", 3), (4, "*", 7),
30                (15, "/", 4), (2, "**", 8), (9, "/", 0), (3, "%", 2)]
31
32 for a, op, b in operaciones:
33     resultado = calculadora(a, op, b)
34     print(f"  {a:>4} {op:>2} {b:<4} = {resultado}")
35
36 # --- Parte 2: Clasificación de figuras geométricas ---
37 print("\n" + "=" * 45)
38 print("  CLASIFICACIÓN DE FIGURAS")
39 print("=" * 45)
40
41 def clasificar_figura(lados):
42     match lados:
43         case 3:
44             return "Triángulo"
45         case 4:
46             return "Cuadrilátero (cuadrado, rectángulo...)"
47         case 5:
48             return "Pentágono"
49         case 6:
50             return "Hexágono"
51         case n if n > 6:
52             return f"Polígono de {n} lados"
53         case _:
54             return "Figura no válida"

```

```

55
56 for lados in [3, 4, 5, 6, 8, 12, 1]:
57     print(f"  {lados} lados -> {clasificar_figura(lados)}")
58
59 # --- Parte 3: Sistema de menú con match-case ---
60 print("\n" + "=" * 45)
61 print("  SIMULACIÓN DE MENÚ DE SISTEMA")
62 print("=" * 45)
63
64 comandos = ["inicio", "ayuda", "salir", "config", "datos", "desconocido"]
65
66 def ejecutar_comando(cmd):
67     match cmd:
68         case "inicio":
69             return "Sistema iniciado correctamente."
70         case "ayuda":
71             return "Comandos: inicio | ayuda | salir | config | datos"
72         case "salir":
73             return "Cerrando sesión..."
74         case "config":
75             return "Abriendo configuración del sistema."
76         case "datos":
77             return "Cargando módulo de datos."
78         case _:
79             return f"Comando '{cmd}' no reconocido. Escribe 'ayuda'."
80
81 for cmd in comandos:
82     print(f"  > {cmd:<15} -> {ejecutar_comando(cmd)}")

```

Listing 5: ejemplo5.py — match/case

Parte 1 — Calculadora. La función recibe el operador como *cadena* y lo compara con cada **case**. El caso `/` incluye una guardia interna (`if b == 0`) para la división entre cero. El **case** `_`: actúa como comodín capturando cualquier operador no soportado.

Parte 2 — Clasificación de figuras. Los primeros casos son *literales enteros*. El caso **case n if n > 6**: usa una guarda: primero vincula el valor a la variable `n` y luego verifica la condición. El comodín final maneja entradas inválidas.

Parte 3 — Menú de comandos. Simula un *despachador* de comandos de texto. El patrón **match/case** es más legible que una cadena de **if/elif** cuando se despacha sobre un mismo valor con muchas ramas posibles.

Salida esperada

```

10 + 5    = 15
2 ** 8    = 256
9 / 0     = Error: division entre cero
3 lados -> Triangulo
8 lados -> Poligono de 8 lados
> inicio  -> Sistema iniciado correctamente.
> desconocido -> Comando 'desconocido' no reconocido.

```

5 Resumen comparativo de estructuras

Estructura	Uso principal	Cuándo elegirla
for anidados	Recorrer matrices 2D y patrones	Se conoce el número de iteraciones
random	Simulación, pruebas	Se necesita variabilidad o aleatoriedad
Ordenamiento	Organizar colecciones	Búsqueda eficiente posterior
while	Condición de parada variable	No se sabe cuántas iteraciones se necesitan
while True	Bucle indefinido controlado	Menús, juegos, servidores
match/case	Despacho por valor	Muchas ramas sobre una misma variable

Cuadro 1: Comparativa de estructuras de control cubiertas en los ejemplos

Para ejecutar los ejemplos

```
# Dentro de la carpeta Ejemplos/
/home/aboudonji/miniforge3/envs/research/bin/python ejemplo1.py
/home/aboudonji/miniforge3/envs/research/bin/python ejemplo2.py
/home/aboudonji/miniforge3/envs/research/bin/python ejemplo3.py
/home/aboudonji/miniforge3/envs/research/bin/python ejemplo4.py
/home/aboudonji/miniforge3/envs/research/bin/python ejemplo5.py
```

Universidad Anáhuac México — Facultad de Ingeniería
 Materia: Algoritmos y Programación en Python
 Prof. D.Sc. Aboud Barsekh-Onji — aboud.barsekh@anahuac.mx
 Ciclo 2025-2026